



S.A.F.E

Sistema de Autorização e
Fluxo Escolar

Joel João de Araujo Neto



Sumário

1. Visão geral da aplicação	1
1.1 Problema resolvido	1
1.2 Módulos principais.....	1
2. Setup inicial.....	1
2.1 PHP	1
2.2 Composer	1
2.3 Node.js e NPM	2
2.4 Banco de dados	2
2.5 Variáveis de ambiente	2
3. Criação do projeto do zero	2
3.1 Criar Laravel	2
3.2 Instalar autenticação e painel	2
3.3 Dependências front-end	2
3.4 Configuração de idioma	2
3.5 Configuração do painel Filament.....	3
4. Banco de dados, migrations e relacionamentos	3
4.1 Relacionamento professor-turma.....	3
4.2 Comandos de banco	3
5. Models, seeders e dados de demonstração.....	4
5.1 Models.....	4
5.2 Seeders obrigatórios	4
6. Autenticação, permissões e policies	4
6.1 Policies implementadas	4
7. Filament e front-end administrativo.....	5
7.1 UX/UI	5
8. Regras de negócio e fluxos digitais	5
8.1 Entrada de aluno atrasado	5
8.2 Saída antecipada	5
8.3 Status de autorização	6
9. Notificações, mensagens e eventos internos	6
9.1 Serviço SafeNotifier	6
10. Testes, validação e deploy	6
10.1 Testes automatizados	6
10.2 Qualidade de código	6
10.3 Checklist de validação	7
10.4 Deploy	7
11. Apêndice: comandos de referência.....	7
12. Melhorias futuras	7

1. Visão geral da aplicação

O SAFE foi criado para substituir autorizações em papel por um fluxo digital rastreável. A aplicação centraliza alunos, turmas, cursos, funcionários, autorizações, ocorrências, mensagens e notificações internas no painel administrativo.

- AQV registra e acompanha solicitações de entrada e saída.
- Professores recebem autorizações das turmas vinculadas e registram leitura, aprovação, recusa ou observações.
- Portaria acompanha liberações e confirma saídas quando necessário.
- Admin gerencia funcionários, permissões, cursos, turmas e dados operacionais.

1.1 Problema resolvido

Em processos manuais, autorizações podem se perder, atrasar ou não deixar evidências suficientes sobre quem solicitou, validou e finalizou a movimentação do aluno. O SAFE organiza esse fluxo em etapas digitais, com auditoria e notificações persistidas.

1.2 Módulos principais

Módulo	Responsabilidade
Funcionários	CRUD administrativo de usuários dos setores AQV, Portaria, Professores e Admin.
Cursos e turmas	Organização acadêmica e vínculo de professores a múltiplas turmas.
Alunos	Cadastro dos estudantes e associação com turmas.
Autorizações	Entrada tardia e saída antecipada com leitura, aprovação, recusa e encerramento.
Ocorrências	Registro de situações escolares com status, observações e histórico.
Notificações	Alertas internos persistidos com status de leitura e badges.
Mensagens	Comunicação interna por usuário ou setor.
Históricos	Auditoria de ações de autorizações e ocorrências.

2. Setup inicial

2.1 PHP

Instale PHP 8.3 ou superior. Em ambientes Windows com Laragon, selecione a versão PHP 8.3 no menu do Laragon.

```
php -v
```

2.2 Composer

Composer é o gerenciador de dependências PHP usado para instalar Laravel, Filament e pacotes auxiliares.

```
composer -V
```

2.3 Node.js e NPM

Node e NPM são usados para compilar CSS/JavaScript via Vite.

```
node -v  
npm -v
```

2.4 Banco de dados

O projeto foi preparado para MySQL em desenvolvimento local e SQLite em memória nos testes. Para MySQL, crie um banco chamado safe.

```
CREATE DATABASE safe CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

2.5 Variáveis de ambiente

```
cp .env.example .env  
php artisan key:generate
```

Exemplo de configuração local MySQL:

```
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3308  
DB_DATABASE=safe  
DB_USERNAME=root  
DB_PASSWORD=
```

3. Criação do projeto do zero

3.1 Criar Laravel

```
composer create-project laravel/laravel SAFE  
cd SAFE
```

3.2 Instalar autenticação e painel

```
composer require laravel/breeze --dev  
php artisan breeze:install blade  
composer require filament/filament spatie/laravel-permission  
php artisan filament:install --panels
```

3.3 Dependências front-end

```
npm install  
npm run build
```

3.4 Configuração de idioma

Defina APP_LOCALE, APP_FALLBACK_LOCALE e APP_FAKER_LOCALE como pt_BR no arquivo .env. Também é recomendável manter APP_TIMEZONE como America/Sao_Paulo.

3.5 Configuração do painel Filament

O painel principal fica em /admin. A classe AdminPanelProvider define nome SAFE, logo, tema, grupos de navegação, login customizado, recursos descobertos automaticamente e notificações de banco com polling.

4. Banco de dados, migrations e relacionamentos

Tabela	Campos principais	Relacionamentos
users	name, email, password, sector, is_active	roles Spatie; muitas turmas como professor
courses	name, slug, description	tem muitas turmas
classrooms	name, course_id, course, teacher_id	pertence a curso; tem alunos; muitos professores
classroom_teacher	classroom_id, user_id	pivot entre turmas e professores
students	name, registration, classroom_id	pertence a turma; tem autorizações e ocorrências
authorization_types	name	Entrada ou Saída
authorizations	student_id, type, status, reason, event_at, responsável, observações	pertence a aluno, tipo, solicitante e processador
authorization_audits	authorization_id, user_id, action, note	histórico de ações
occurrences	student_id, registered_by, description, status, observations	ocorrências escolares
occurrence_audits	occurrence_id, user_id, action, note	histórico de ocorrências
notifications	type, notifiable, data, read_at	notificações persistidas Laravel
internal_messages	sender, recipient, recipient_role, subject, body, read_at	mensagens internas

4.1 Relacionamento professor-turma

O SAFE usa uma relação muitos-para-muitos entre professores e turmas. Essa decisão permite que Eduardo acesse DS e Eletroeletrônica ao mesmo tempo em que Samuel fica limitado à Eletroeletrônica e Bruno ao Desenvolvimento de Sistemas.

4.2 Comandos de banco

```
php artisan migrate
php artisan db:seed
# Para recriar ambiente de demonstração
php artisan migrate:fresh --seed
```

5. Models, seeders e dados de demonstração

5.1 Models

- User implementa FilamentUser para controlar acesso ao painel com canAccessPanel.
- User possui teachingClassrooms para o vínculo muitos-para-muitos com turmas.
- Classroom pertence a Course, possui Students e Teachers.
- Authorization concentra os métodos markAsRead, approve, deny e finish.
- Occurrence concentra close e registra auditoria.
- InternalMessage notifica destinatários ao ser criada.

5.2 Seeders obrigatórios

Os seeders criam automaticamente cursos, turmas, alunos, tipos de autorização, perfis e usuários de teste.

Professor	E-mail	Escopo de acesso
Eduardo	eduardo@safe.com	Desenvolvimento de Sistemas e Eletroeletrônica
Samuel	samuel@safe.com	Somente Eletroeletrônica
Bruno	bruno@safe.com	Somente Desenvolvimento de Sistemas

Cada um dos dois cursos recebe três turmas, e cada turma recebe cinco alunos com nomes e matrículas realistas.

6. Autenticação, permissões e policies

A autorização combina Spatie Laravel Permission, policies do Laravel e filtros de consulta nos Resources do Filament.

Perfil	Acesso principal
Admin	Acesso completo, incluindo CRUD de funcionários e exclusões administrativas.
AQV	Gerencia autorizações, ocorrências, cursos, turmas e alunos.
Professor	Visualiza alunos, turmas, autorizações e ocorrências apenas das turmas vinculadas.
Portaria	Acompanha autorizações, confirma saídas e usa mensagens/notificações.

6.1 Policies implementadas

- UserPolicy limita gestão de funcionários ao Admin.
- StudentPolicy e ClassroomPolicy aplicam escopo por turma.
- AuthorizationPolicy permite visualização por setor e restringe edição estrutural a Admin/AQV.
- OccurrencePolicy controla visualização e encerramento.
- InternalMessagePolicy limita visualização a remetente, destinatário, setor ou Admin.
- SystemNotificationPolicy garante que cada usuário veja apenas suas notificações.

7. Filament e front-end administrativo

O front-end principal é o painel Filament. A navegação é organizada em Operação escolar, Comunicação, Cadastros acadêmicos e Administração.

Resource	Tela
Employees	Funcionários, setor, status ativo e turmas vinculadas ao professor.
Courses	Cursos técnicos.
Classrooms	Turmas, curso, professores vinculados e quantidade de alunos.
Students	Alunos, matrícula, turma e curso.
Authorizations	Autorizações digitais e ações do fluxo.
Occurrences	Ocorrências com status e encerramento.
Notifications	Notificações persistidas e status de leitura.
InternalMessages	Mensagens por destinatário ou setor.
AuthorizationAudits e OccurrenceAudits	Históricos somente leitura.

7.1 UX/UI

- Login customizado com identidade SAFE e atalhos de usuários de teste.
- Badges de notificações e mensagens não lidas.
- Filtros por status, tipo de autorização e turma.
- Ações de fluxo diretamente na tabela para reduzir cliques.
- Campos de observação separados para AQV, professor e portaria.

8. Regras de negócio e fluxos digitais

8.1 Entrada de aluno atrasado

1. Portaria identifica ou encaminha o aluno atrasado.
2. AQV cria autorização de Entrada com aluno, horário, motivo, responsável e observações.
3. Professores vinculados à turma recebem notificação.
4. Professor confirma leitura e registra observação se necessário.
5. Professor aprova ou recusa a autorização.
6. Sistema grava status, usuário responsável e auditoria.

8.2 Saída antecipada

7. AQV cria autorização do tipo Saída.
8. Professor recebe notificação por pertencer à turma do aluno.
9. Professor confirma leitura e aprova/libera ou recusa.
10. Quando aprovado, a portaria recebe notificação de liberação.
11. Portaria confirma a saída e finaliza o fluxo.
12. Sistema registra horários, responsáveis, status e histórico completo.

8.3 Status de autorização

Status	Significado
pending	Aguardando validação.
approved	Aprovada ou liberada pelo professor/AQV.
denied	Recusada com justificativa.
finished	Fluxo encerrado pela AQV ou portaria.

9. Notificações, mensagens e eventos internos

As notificações usam a tabela notifications do Laravel. Cada notificação possui payload JSON com título, mensagem, URL, categoria e cor. O painel Filament exibe notificações de banco com polling configurado.

- Criação de autorização notifica professores da turma.
- Leitura notifica o solicitante.
- Aprovação de saída notifica a portaria.
- Recusa notifica solicitante e professores envolvidos.
- Finalização notifica solicitante e professores.
- Ocorrência notifica AQV/Admin e professores da turma.
- Mensagem interna notifica destinatário direto ou setor.

9.1 Serviço SafeNotifier

SafeNotifier centraliza as regras de destinatários. Essa camada evita duplicação de lógica nos models e nas telas, e facilita a evolução futura para broadcast em tempo real.

10. Testes, validação e deploy

10.1 Testes automatizados

A suíte cobre autenticação, seeders, escopo de professores, fluxos de autorização, ocorrências, mensagens, notificações e renderização das principais telas do painel.

```
php artisan test
```

10.2 Qualidade de código

```
vendor/bin/pint --test  
npm run build
```


10.3 Checklist de validação

- Executar migrate:fresh --seed sem erros.
- Confirmar login com Admin, AQV, Portaria e professores.
- Validar que Eduardo vê DS e Eletroeletrônica.
- Validar que Samuel vê apenas Eletroeletrônica.
- Validar que Bruno vê apenas Desenvolvimento de Sistemas.
- Criar autorização de Entrada e confirmar leitura/aprovação.
- Criar autorização de Saída e finalizar pela portaria.
- Criar ocorrência e verificar notificação e histórico.
- Enviar mensagem interna e marcar como lida.
- Verificar badges e listagens de notificações.

10.4 Deploy

13. Configurar servidor com PHP 8.3, extensões PHP, Composer, Node e banco MySQL.
14. Clonar o repositório e configurar .env de produção.
15. Executar composer install --no-dev --optimize-autoloader.
16. Executar npm ci e npm run build.
17. Executar php artisan migrate --force.
18. Configurar storage:link, permissões de storage e cache.
19. Executar php artisan config:cache, route:cache e view:cache.
20. Apontar o servidor web para public/.

11. Apêndice: comandos de referência

```
composer install
npm install
cp .env.example .env
php artisan key:generate
php artisan migrate:fresh --seed
npm run build
php artisan serve
php artisan test
vendor/bin/pint --test
```

12. Melhorias futuras

- Substituir polling por WebSockets com Laravel Reverb ou serviço compatível.
- Criar relatórios por curso, turma, período, motivo e setor.
- Adicionar QR Code para conferência de autorização na portaria.
- Adicionar cadastro de responsáveis legais e contato de emergência.
- Exportar autorizações e ocorrências em PDF.
- Criar dashboard individual para AQV, professores e portaria.
- Integrar com sistemas acadêmicos externos.